

Exact Domination of Lecture Timings: An Efficient Algorithm

Your Name (Roll Number)

Abstract

A supervisor wants to choose a subset of courses so that every lecture overlaps with *exactly one* lecture of the chosen subset (a chosen lecture overlaps itself). Lectures are daily and may run past midnight, so they are arcs on a 24-hour circle. We show that the chosen lectures must be pairwise disjoint and that a set is valid if and only if every pair of cyclically consecutive chosen lectures satisfies three local conditions. This turns the problem into finding a closed chain around the circle, which we solve by dynamic programming in $O(n^3)$ time and $O(n^2)$ space, in place of the $O(2^n)$ time of straightforward backtracking. A correctness proof and a complexity analysis are given.

Implementation: <https://github.com/deveshk-boop/daa-project-q17> (see the `README.md` there for build and run instructions).

1 Problem Statement

There are n courses, and course i has one lecture per day, from a start time s_i to an end time e_i given as HH:MM. A lecture may cross midnight (e_i earlier than s_i). Two lectures *overlap* if they are in progress together at some moment of the day. We must select a subset S of the lectures such that every lecture overlaps with *exactly one* lecture in S , counting the lecture itself if it is in S . If no such S exists, the algorithm must report this.

Model and assumptions

Time is measured in minutes, so the day is a circle of length 1440 (we write $M = 1440$ when it appears in formulas). Lecture i is the half-open arc

$$A_i = \{s_i, s_i + 1, \dots, s_i + \ell_i - 1\} \pmod{M},$$

where the length is $\ell_i = (e_i - s_i) \bmod M$. If $s_i = e_i$, we set $\ell_i = 1$ (a one-minute lecture); the statement does not define this case, so this is an assumption. Since arcs are half-open, lectures that merely touch (one ends exactly when the next starts) do *not* overlap.

Definition 1. Let $d(x, y) = (y - x) \bmod M$ be the forward (clockwise) distance from time x to time y . A time p lies in A_i iff $d(s_i, p) < \ell_i$. A set S of lectures is *valid* if for every lecture c ,

$$|\{a \in S : A_a \cap A_c \neq \emptyset\}| = 1.$$

2 Structural Results

Lemma 1 (Overlap test). *Two arcs A_i, A_j intersect if and only if $s_i \in A_j$ or $s_j \in A_i$.*

Proof. If $s_i \in A_j$ then $s_i \in A_i \cap A_j$, and symmetrically for $s_j \in A_i$. Conversely, let $p \in A_i \cap A_j$ and assume without loss of generality that $d(s_j, p) \leq d(s_i, p)$. Both distances are less than M , so $d(s_i, s_j) = d(s_i, p) - d(s_j, p) \leq d(s_i, p) < \ell_i$, hence $s_j \in A_i$. $\square$

Thus the overlap table can be computed with two membership tests per pair.

Lemma 2 (Chosen lectures are disjoint). *If S is valid, then the arcs of S are pairwise disjoint.*

Proof. Take $a \in S$. The lecture a overlaps itself, and it must overlap exactly one member of S , so it overlaps no other member of S . $\square$

A valid set with a single lecture is easy to describe: $S = \{a\}$ is valid iff A_a intersects every arc. From now on, let S be a set of $k \geq 2$ pairwise disjoint arcs. Listing them by start time clockwise gives a cyclic order $x_0, x_1, \dots, x_{k-1}$, with indices taken modulo k . We say x_{i+1} *follows* x_i .

Definition 2 (Gap). For $a, b \in S$ with b following a , the *gap* is $\text{gap}(a, b) = \{p : d(e_a, p) < d(e_a, s_b)\}$, the times from the end of a up to (not including) the start of b .

Definition 3 (Compatibility $F(a, b)$). For distinct lectures a, b , let $F(a, b)$ hold iff all three conditions hold:

(F1) $A_a \cap A_b = \emptyset$;

(F2) no lecture c overlaps both a and b ;

(F3) no lecture c that overlaps neither a nor b has its start s_c in $\text{gap}(a, b)$.

Lemma 3 (Contiguity). *Let S be a set of $k \geq 2$ pairwise disjoint arcs. If an arc A_c overlaps two arcs of S , then it overlaps two cyclically consecutive arcs of S .*

Proof. Suppose A_c contains a point $u \in A_{x_i}$ and a point $v \in A_{x_j}$ with $x_i \neq x_j$. Since A_c is a connected arc of the circle, it contains one of the two circular paths from u to v ; take the clockwise path from u (the other case is symmetric). Leaving A_{x_i} and moving clockwise, the first arc of S that the path enters is x_{i+1} , because x_{i+1} is the next arc by start time and arcs of S are disjoint. So A_c contains a point of $A_{x_{i+1}}$ and overlaps both x_i and x_{i+1} . $\square$

Lemma 4 (Local characterisation). *Let S be a set of $k \geq 2$ pairwise disjoint arcs in cyclic order $x_0, \dots, x_{k-1}$. Then S is valid if and only if $F(x_i, x_{i+1})$ holds for all i (indices modulo k).*

Proof. ($\Rightarrow$) Let S be valid and take a consecutive pair $a = x_i, b = x_{i+1}$. (F1) holds because S is pairwise disjoint. For (F2), a lecture overlapping both a and b would overlap two members of S , contradicting validity. For (F3), suppose c overlaps neither a nor b and $s_c \in \text{gap}(a, b)$. Since $s_b \notin A_c$, we get $\ell_c \leq d(s_c, s_b)$, so $A_c \subseteq [s_c, s_b] \subseteq \text{gap}(a, b)$. No arc of S meets the gap: if $x \in S$ contained a point p of the gap, walk backward from p inside A_x . No start time of S lies in $[e_a, s_b]$, so the walk would have to pass through the last point of A_a , forcing $x = a$; but A_a does not contain p . Hence c overlaps no member of S , contradicting validity. So (F3) holds.

($\Leftarrow$) Assume F holds for all consecutive pairs and let c be any lecture. *At most one:* if c overlapped two members of S , then by Lemma 3 it would overlap some consecutive pair, violating (F2). *At least one:* suppose c overlaps no member of S . Then s_c lies in no arc of S , so it lies in some gap $\text{gap}(x_i, x_{i+1})$ (the circle minus the arcs of S is the union of the gaps), and c overlaps neither x_i nor x_{i+1} ; this violates (F3). Hence c overlaps exactly one member of S . $\square$

Reduction to a closed chain

Fix a lecture a_0 and define $\text{pos}(x) = d(s_{a_0}, s_x)$, the clockwise offset of the start of x from the start of a_0 .

Lemma 5 (Chains). *Let $a_0 \in S$, where S is a set of $k \geq 2$ lectures.*

1. If S is valid, listing S in cyclic order starting at a_0 gives a sequence $a_0 = x_0, x_1, \dots, x_m$ ($m = k-1$) with $0 = \text{pos}(x_0) < \text{pos}(x_1) < \dots < \text{pos}(x_m) < M$, $F(x_i, x_{i+1})$ for $0 \leq i < m$, and $F(x_m, a_0)$.
2. Conversely, if a sequence $a_0 = x_0, \dots, x_m$ with $m \geq 1$ has all these properties, then $\{x_0, \dots, x_m\}$ is valid.

Proof. (1) By Lemma 2 the arcs are pairwise disjoint, so their cyclic order by start time is well defined, and reading it from a_0 gives strictly increasing offsets. The F conditions follow from Lemma 4.

(2) Write $p_i = \text{pos}(x_i)$ and ℓ_i for the length of x_i . We claim $p_i + \ell_i \leq p_{i+1}$ for $i < m$, and $p_m + \ell_m \leq M$. If $p_i + \ell_i > p_{i+1}$ for some $i < m$, then $p_i < p_{i+1} < p_i + \ell_i$, so the start of x_{i+1} lies in A_{x_i} , contradicting (F1) for (x_i, x_{i+1}) . If $p_m + \ell_m > M$, then the start of a_0 (offset $M \equiv 0$) lies in A_{x_m} , contradicting (F1) for (x_m, a_0) . Thus the arcs occupy the disjoint intervals $[p_i, p_i + \ell_i) \subseteq [0, M)$ in increasing order. Hence they are pairwise disjoint, their cyclic order is $x_0, \dots, x_m$, and the pairs checked by F are exactly the consecutive pairs. By Lemma 4, the set is valid. $\square$

3 The Algorithm

Idea. Handle the single-lecture case directly. Otherwise, build the table $F(\cdot, \cdot)$ and look for a closed chain around the circle. Since the lecture numbered 1 must overlap some chosen lecture, every valid set contains a lecture overlapping it, so it suffices to try each such lecture as the starting point a_0 .

Algorithm 1 EXACTDOMINATION($A_1, \dots, A_n$)

```

1: build  $O[i][j] \leftarrow [A_i \cap A_j \neq \emptyset]$  for all  $i, j$  (Lemma 1);  $O[i][i] \leftarrow \mathbf{true}$ 
2: for  $a = 1$  to  $n$  do ▷ single-lecture case
3:   if  $O[a][c]$  for all  $c$  then return  $\{a\}$ 
4:   end if
5: end for
6: build  $F[a][b]$  for all  $a \neq b$  using COMPUTEF
7: for  $a_0 = 1$  to  $n$  with  $O[1][a_0]$  do
8:    $chain \leftarrow \text{SEARCHCHAIN}(a_0)$ 
9:   if  $chain$  is not empty then return  $chain$ 
10:  end if
11: end for
12: return “no valid subset exists”

```

4 Correctness

Lemma 6. Inside SEARCHCHAIN, any chain $a_0 = x_0, x_1, \dots$ built by the dynamic program satisfies $\text{pos}(x_1) < \text{pos}(x_2) < \dots$ strictly, and $\text{pos}(x_1) > 0$.

Proof. The program only extends a chain by an element that is later in the sorted order, so offsets never decrease. Two distinct lectures with the same start time overlap (each starts inside the other), so (F1) fails for them, and the program never links them. Hence offsets strictly increase, and since any lecture with $\text{pos} = 0$ has the same start as a_0 and overlaps it, x_1 has $\text{pos} > 0$. $\square$

Algorithm 2 COMPUTEF(a, b)

```
1: if  $O[a][b]$  then return false
2: end if ▷ (F1)
3:  $g \leftarrow d(e_a, s_b)$  ▷ length of the gap
4: for  $c = 1$  to  $n$  do
5:   if  $O[c][a]$  and  $O[c][b]$  then return false
6:   end if ▷ (F2)
7:   if not  $O[c][a]$  and not  $O[c][b]$  and  $d(e_a, s_c) < g$  then
8:     return false ▷ (F3)
9:   end if
10: end for
11: return true
```

Algorithm 3 SEARCHCHAIN(a_0)

```
1: order  $\leftarrow$  all lectures except  $a_0$ , sorted by  $\text{pos}(x) = d(s_{a_0}, s_x)$ 
2: reach[x]  $\leftarrow$  false and prev[x]  $\leftarrow$  none for all  $x$ 
3: for  $i = 1$  to  $n - 1$  do
4:    $x \leftarrow \text{order}[i]$ 
5:   if  $F[a_0][x]$  then
6:     reach[x]  $\leftarrow$  true; prev[x]  $\leftarrow a_0$ 
7:   else
8:     for  $j = 1$  to  $i - 1$  do ▷ a predecessor earlier in the order
9:        $y \leftarrow \text{order}[j]$ 
10:      if reach[y] and  $F[y][x]$  then
11:        reach[x]  $\leftarrow$  true; prev[x]  $\leftarrow y$ ; break
12:      end if
13:    end for
14:  end if
15:  if reach[x] and  $F[x][a_0]$  then ▷ the chain closes around the circle
16:    return  $x, \text{prev}[x], \text{prev}[\text{prev}[x]], \dots, a_0$ 
17:  end if
18: end for
19: return empty
```

Lemma 7 (SEARCHCHAIN). *SEARCHCHAIN(a_0) returns a non-empty list if and only if there is a sequence $a_0 = x_0, x_1, \dots, x_m$ ($m \geq 1$) with strictly increasing offsets, $F(x_i, x_{i+1})$ for $i < m$, and $F(x_m, a_0)$. The returned list is such a sequence.*

Proof. By induction over the sorted order, reach[x] is true iff some chain from a_0 to x exists with F on every step and increasing offsets: the base case is $F[a_0][x]$, and otherwise the program tries every earlier y that is reachable. The chain is returned exactly at the first reachable x with $F[x][a_0]$. Following the prev pointers reconstructs such a chain, using Lemma 6 for strictness. $\square$

Theorem 1. *EXACTDOMINATION outputs a valid subset if one exists, and reports that none exists otherwise.*

Proof. Soundness. If a single lecture is output, it overlaps every lecture, so it is valid. If a chain is output, then Lemmas 7 and 5(2) show the chosen set is valid.

Completeness. Suppose a valid set S exists. If $|S| = 1$, the single-lecture test succeeds. Otherwise $|S| \geq 2$. Lecture 1 must overlap a member of S ; call it a_0 , so $O[1][a_0]$ holds and a_0 is tried by the main loop. By Lemma 5(1), S listed in cyclic order from a_0 is a chain of

the required form, so by Lemma 7, `SEARCHCHAIN( $a_0$ )` (or an earlier call) returns a non-empty chain, which is valid by soundness. Therefore the algorithm never wrongly reports that no valid set exists. $\square$

5 Complexity Analysis

Let n be the number of lectures.

- **Overlap table.** $O(1)$ per pair, so $O(n^2)$ time and $O(n^2)$ space.
- **Single-lecture test.** Checking each lecture against all others takes $O(n^2)$.
- **Table F .** `COMPUTE $F$` scans all n lectures, so $O(n)$ per pair and $O(n^3)$ over all $n(n-1)$ pairs; the table takes $O(n^2)$ space.
- **One call of `SearchChain`.** Sorting costs $O(n \log n)$. The double loop performs $O(n^2)$ table lookups, each $O(1)$, and reconstruction is $O(n)$. The total is $O(n^2)$.
- **Main loop.** At most n starting lectures, so $O(n) \cdot O(n^2) = O(n^3)$.

Overall the running time is

$$O(n^2) + O(n^3) + O(n^3) = O(n^3),$$

and the space is $O(n^2)$ for the tables O and F (the per-call arrays use $O(n)$). Reading the input takes $O(n)$. This is polynomial, in contrast to the $O(2^n)$ worst case of the backtracking baseline, which tries every subset.

Remark 1. We do not claim that $O(n^3)$ is optimal. It is chosen because each step is easy to prove correct; the table F and the search could likely be sped up by exploiting the interval structure, but that is not needed here.

6 Implementation and Testing

The algorithm is implemented in C++ (`src/exact_domination.cpp`). Input is n followed by n lines `HH:MM HH:MM`. The output is either the selected lecture numbers or `No valid subset exists`. The repository also contains the original backtracking solution (`src/brute_force.cpp`) and a file of hand-verified test cases (`tests/all_testcases.txt`), summarised below.

A ring of k lectures, each overlapping only its two neighbours, has a valid subset exactly when $3 \mid k$: each chosen lecture covers itself and its two neighbours, so the lectures must split into blocks of three. This explains the outcomes of cases 9–11.

7 Conclusion

We characterised valid selections as pairwise disjoint sets whose cyclically consecutive members satisfy three local conditions, reduced the search to a closed chain around the day, and solved it in $O(n^3)$ time and $O(n^2)$ space, with a proof of correctness. The stated assumptions are half-open lecture intervals (touching lectures do not overlap) and one-minute duration for lectures with equal start and end time.

#	Scenario	Expected output
1	single lecture	1
2	one lecture overlaps all others	1
3	identical lectures	any single lecture
4	all lectures disjoint	all lectures
5	two separate groups	one from each group
6	chain 1–2–3–4	1 4
7	touching lectures	1 2
8	lectures crossing midnight	1 3 or 2 3
9	ring of 6 lectures	1 4, 2 5 or 3 6
10	ring of 5 lectures	no valid subset
11	ring of 4 lectures	no valid subset

Table 1: Test cases covering simple, boundary, wrap-around and infeasible inputs.